Phase 4 developer handoff: evidence intelligence

This implementation reuses immutable `AnalysisRun.result`, completed runtime reports/events, conservative mapping, coverage and fusion. PostgreSQL is authoritative. A separate persistent Neo4j database is a reproducible projection. No second static scanner, OTLP receiver or model-dispatch path is introduced.

Modules and persistence

Module	Responsibility
`backend/app/evidence/models.py`	Tenant-owned architecture versions, canonical evidence, facts, projection outbox and discovery progress
`schemas.py`	Bounded version input, explicit domain review and optimistic discovery updates
`builder.py`	Pure version-to-entity/evidence/fact adapter; deterministic identities, classification and discovery suggestions
`service.py`	Source compatibility, frozen manifests, idempotency, domain reviews, source retention and provenance resolution
`projection.py`	Fixed parameterized Neo4j operations, atomic replay, scope/currentness checks and bounded queries
`routes.py`	Scoped version, evidence, fact, graph, twin, discovery, comparison and export endpoints
`backend/alembic/versions/0005_evidence.py`	Five additive tables; previous analysis/security/runtime records are not rewritten
`frontend/src/evidence/EvidenceWorkspace.tsx`	Version selection, preview, domain review, fact provenance, discovery, persistent graph and twin
`scripts/install_knowledge_graph.py`	Pinned SHA-256-verified local distribution installation
`scripts/knowledge_graph.py`	Authenticated loopback graph setup/start/check, isolated directories and recorded JVM PID

Tables: `architecture_versions`, `architecture_evidence`, `architecture_facts`, `architecture_projections`, `architecture_discovery`. Each inherits tenant identity and uses the existing scoped ORM read/write boundary. Evidence/fact/discovery records reference their version; the version references application, analysis and optional parent. Names and operational status do not replace stable IDs.

`ArchitectureVersion.document` stores the canonical graph, coverage, quality, domains, method observations, collector configuration and explicit unknowns. Evidence/fact records hold authoritative structured provenance. The projection outbox is committed in the same SQL transaction as its source records. Discovery state/history is mutable and deliberately separate from the frozen snapshot/export.

Creation, identity and provenance

Creation accepts one completed static baseline and zero to ten completed runtime sessions from that exact application/baseline. Active, foreign, missing or incompatible records are rejected. Source snapshots are copied, never mutated. Existing expired raw-session merge restrictions remain in force.

The canonical fingerprint includes application/analysis/artifact association, static digest, selected runtime report digests, parent, rule version, domain review and available method-observation digest. Timestamps generated during derivation and display name do not change the identity. Tenant + fingerprint deterministically produces the version UUID; internal UUIDs are namespaced by that version. Repeated creation returns the existing version. SQL uniqueness and the supported single-process lock enforce idempotency. Historical

payloads are not backfilled automatically on startup; explicitly capturing an old completed baseline is the supported idempotent backfill operation.

A reviewed domain configuration must retain the existing domain IDs, assign the full static class set exactly once, remain nonempty and have an actor/reason. It is recorded in the version manifest plus USER_CONFIRMED evidence. Names/memberships can change without changing domain identity. Original findings/metrics remain baseline-scoped; no downstream decision/assessment is silently reused as if recomputed. Versions with differing domain-review scopes cannot be compared through the observation-delta endpoint.

The pure builder publishes first-class STATIC, RUNTIME, COVERAGE, METRIC, DATABASE, DERIVED and USER_CONFIRMED evidence. Each record carries source/target, observation, collector, derivation timestamp, confidence, analysis ID, applicable session IDs and bounded references. Runtime edge metrics preserve source timestamps. Static evidence has no runtime-session source attribution. Existing findings/cycles are imported with DERIVED provenance; this does not implement Phase 5 finding/decision intelligence.

Facts represent directed internal fused dependencies with STATIC_ONLY, DYNAMIC_ONLY or STATIC_AND_DYNAMIC classification, exact supporting evidence IDs, static types and observed count. `conflict_ids` is reserved for explicit contradictory evidence; absence of a runtime observation is not manufactured into a contradiction. Duplicate-span conflicts remain runtime quality metadata because they do not identify a trustworthy contradictory architecture assertion. No manual unsupported fact publication endpoint exists.

Signature-level method observations come from retained sanitized events and the Phase 3 mapper. After raw expiry they are unknown; aggregate report counts are not expanded into invented method identities. Existing canonical exports remain immutable through expiry. Provenance resolution checks raw source availability at read time and marks unavailable references explicitly.

Projection, failure and rebuild

Neo4j Community **5.26.30** uses `.runtime/neo4j-community-5.26.30` binaries and `.data/knowledge-graph` persistent data/config/logs. This is separate from jQAssistant's embedded per-analysis databases. Authentication is enabled, HTTP/HTTPS connectors disabled, Bolt bound to `127.0.0.1:17687`, usage reporting disabled, heap bounded at 512 MB and page cache at 128 MB. The generated local password is read through a file reference and is never copied into evidence, logs or API responses. Protect that file with appropriate workstation access controls; managed vaults/encrypted deployment are later deployment work.

Settings: `KNOWLEDGE_GRAPH_ENABLED`, `KNOWLEDGE_GRAPH_URI`, `KNOWLEDGE_GRAPH_USER`, `KNOWLEDGE_GRAPH_PASSWORD_FILE` or `KNOWLEDGE_GRAPH_PASSWORD`. This local adapter only accepts literal loopback `bolt://` addresses with an explicit port and no embedded credentials, query or fragment. Routing and remote graph destinations are rejected. The existing Neo4j Python dependency is reused.

Projection uses fixed `ArchIntelEntity`, `ArchIntelProjection` and `ARCH_LINK` schema names. Unique entity IDs and projection scopes plus a scope index are provisioned idempotently. Every graph read/write is fenced by `tenant_id:version_id`; no client Cypher or labels are interpolated. A rebuild transaction deletes/recreates only that version's projection, batches nodes/relationships, and writes its digest/count marker atomically. Other versions and scanner stores are untouched.

The background worker runs every 30 seconds, processes at most two pending/failed jobs per sweep, and automatically retries fewer than ten attempts. Manual rebuild can retry afterward. Failures persist a safe error class, not provider URLs, credentials or payload text. If Neo4j commits and the SQL process crashes before acknowledgement, replay produces the same graph. Canonical SQL records remain available during graph failure.

Graph reads compare the version content digest and actual node/relationship counts with the marker; missing/stale/incomplete projections return 409, unavailable connections return 503. This is integrity/currentness checking of the managed projection, not protection against a malicious database administrator rewriting graph payloads and markers. APPLIED is the last successful projection state; the twin does not misrepresent it as a current liveness probe.

Version/discovery mutation request bodies are capped at 2 MiB after authentication. Version creation is bounded at 10,000 classes, 30,000 total internal/external static edges, 100,000 canonical nodes and 250,000 relationships. Fact/evidence/discovery pages are at most 200 records. Graph reads return at most 500 nodes and 1,000 induced relationships; the UI requests 150 nodes. Neighborhoods are one hop and scoped. Full canonical exports are explicit bounded-version exports. Large-population production latency/load benchmarking is not claimed.

HTTP contracts and authorization

Endpoint suffix under `/api/v1`	Purpose
`POST /applications/{app}/architecture-versions/preview`	Validated deterministic preview without persistence
`POST /applications/{app}/architecture-versions`	Idempotent canonical version capture
`GET /applications/{app}/architecture-versions`	Up to 200 permitted versions
`GET /architecture-versions/{id}`	Frozen manifest and counts
`GET /architecture-versions/{id}/twin`	Supported model/coverage/unknowns plus operational projection state
`GET .../facts`, `GET .../evidence`	Paged canonical records; optional fact classification filter
`GET .../facts/{fact}/provenance`	Complete bounded supporting records and current raw-reference availability
`GET .../graph?limit=150&root={optional-node-id}`	Bounded persistent graph/currentness check
`POST .../projection/rebuild`	Retry/replay one version
`GET .../discovery`, `POST .../discovery/{item}`	Prioritized gap queue and revision-checked status/note updates
`GET .../compare/{other-version}`	Compatible observation percentage-point differences
`GET .../export`	Frozen version + graph + canonical evidence/facts

The existing middleware requires ANALYSIS_READ for reads and ANALYSIS_WRITE for mutations/preview. Runtime-containing versions additionally require RUNTIME_READ, including their parents, graph, export and provenance. Tenant enforcement applies to version IDs, facts, discovery IDs and source references. CSRF, workspace audit and source/model gateway protections remain unchanged. These APIs never request source files or call a model.

Discovery supports OPEN/PLANNED/EXERCISED/DISMISSED, a required note and `expected_revision`; stale updates return 409. Histories record actor and timestamp, with 100 updates per item. Supported suggestions cover annotated routes/listeners/schedulers, unobserved static METHOD_CALL pairs, partially observed domains and baseline high-risk findings. A status update does not change evidence or coverage; a successor capture and compatible comparison provide new observations.

Installation and validation

From the project root in PowerShell:

```
.\.runtime\python\tools\python.exe scripts/install_knowledge_graph.py
.\.runtime\python\tools\python.exe scripts/knowledge_graph.py setup
.\.runtime\python\tools\python.exe scripts/knowledge_graph.py start
.\.runtime\python\tools\python.exe scripts/knowledge_graph.py check
```

Set `KNOWLEDGE_GRAPH_ENABLED=true` and `KNOWLEDGE_GRAPH_PASSWORD_FILE=.data/knowledge-graph/password.key` in the local `.env`. The normal launcher then starts/checks the graph; its failure does not prevent core static/canonical API startup. Stop validates the recorded PID, JVM executable and workspace command line before stopping the graph JVM. Initial distribution download needs internet unless its verified archive is already cached. Phase 9 offline packaging, backups and production deployment acceptance remain separate.

Regression commands:

```
$env:ARCHINTEL_RUN_SCANNER_TESTS='1'
$env:ARCHINTEL_RUN_MODEL_TESTS='1'
$env:ARCHINTEL_RUN_POSTGRES_TESTS='1'
$env:ARCHINTEL_RUN_GRAPH_TESTS='1'
$phaseTests='.data/phase4-tests-'+[guid]::NewGuid().ToString('N')
.\.runtime\python\tools\python.exe -m pytest backend/tests -q -p no:cacheprovider --basetemp=$phaseTests
```

From `frontend`, run `npm.cmd test -- --run`, `npm.cmd run build`, and `npm.cmd exec playwright test -- --config=playwright.security.config.ts` with the model/graph flags set. The browser server uses an isolated SQLite workspace; graph versions use distinct test-owned scopes.

`backend/tests/test_evidence.py` includes the real PostgreSQL + Neo4j integration. It reuses the already validated Phase 3 PetClinic trace capture, copies it into a temporary PostgreSQL schema, verifies 52 facts and complete canonical records, API restart, real graph projection/replay, and graph endpoint failure without SQL evidence loss. The test removes only its generated schema/scopes. It writes `docs/phase4-integration-validation.json`; this is reuse of real captured telemetry, not a claim that Phase 4 launched a second fresh Java workload. Regenerate its source with `scripts/validate_phase3_runtime.py` if needed.

Remaining boundaries

One API writer/process remains the supported deployment. No distributed outbox leases, production graph backup/restore, tenant DB-level Neo4j RBAC, graph administrator tamper-proofing, external-provider routing, code edits or autonomous execution are introduced. Domain revisions are immutable version-manifest records with confirmation evidence, not a separate editable domain registry. New domain creation/merge/split identity workflows are not exposed. Twin tests/contracts/data ownership/execution state remain explicitly unknown unless an implemented source supports them. Phase 5 readiness/confidence/decisions/waves and Phase 6+ transformation are not implemented here.

Primary references: Neo4j Windows installation, configuration settings. The pinned distribution manifest/checksum is in `.runtime/neo4j-distribution.json`.